Presentation script

Team LUGER · Project 6: Fault-Tolerant Quadrotor Control · 27 October 2026 · 10 minutes

Who speaks when

Speaker	Slides	What they cover	Time
Batta Venkata Shashank	1–2	Title and objectives	1:20
Nelagonda Pavaneswar	3–4	Why it’s hard, and thrust margin	1:40
Jetty Jithendra	5	Architecture (flight stack)	1:00
Vineel Sai Reddy	6–7	Control reallocation and fault interface	1:55
Hemanth	8–9	After the fault, and deliverables	1:55
Shaik Babu	10–11	Scope, plus airframe and propulsion parts	1:10
Prasad	12–13	Control and safety parts, and closing	1:00
Total			**10:00**

How to use this: each person reads only their section. The orange box at the end of each section is the hand-over line to the next speaker. Times are cumulative from the start of the talk; at a normal pace the whole script takes about 10 minutes.

Speaker 1 · Batta Venkata Shashank

Slides 1–2 · Title and objectives · about 1:20

Slide 1 – Title (0:00–0:20)

Good morning. We're Team LUGER, and we're working on Project 6, fault-tolerant quadrotor control. In one line: we're building a quadrotor that keeps flying its waypoint mission even when one motor is limited to half its normal speed. That's the orange rotor here. We'll cover our objectives, our solution framework, our deliverables and the components we'll use.

Slide 2 – Objectives (0:20–1:20)

We have five objectives, all taken from the project brief. One: build a quadrotor with enough spare thrust that one motor at 50% RPM doesn't bring it down. Two: fly an autonomous waypoint mission with all four motors healthy. That's our baseline, and every result after the fault is compared against it. Three: partway through the mission, one motor is capped at 50%, and our controller is told which one. It has to adapt and keep the drone stable. Four: after the fault, the drone either finishes the mission or flies back and lands safely. Five: every claim we make is backed by flight logs. One thing we're not doing is detecting the fault. The brief says we're told which motor and when, so detection is out of scope.

Hand over: "Next, Nelagonda Pavaneswar will explain why this problem is hard."

Speaker 2 · Nelagonda Pavaneswar

Slides 3–4 · Why it's hard, and thrust margin · about 1:40

Slide 3 – Why it's hard (1:20–2:00)

So why is this hard? Propeller thrust goes with the square of its speed. Half the RPM doesn't give half the thrust; it gives a quarter. A quad also has no spare motor: the four motors handle lift, roll, pitch and yaw together. When one corner gets weaker, the drone tips toward it. And because two motors spin clockwise and two anticlockwise, their torques stop cancelling, so the drone starts turning too. The controller has to handle all three at once.

Slide 4 – Thrust margin (2:00–3:00)

Our first design decision is about the hardware. We size the drone so the weak motor can still hold up its corner. We estimate the drone at about 650 grams, so each motor needs about 163 grams of thrust to hover. The motors we've chosen give around 1.2 kilograms each at full speed. A quarter of that is about 300 grams, which is still more than 163. That works out to a thrust-to-weight ratio of about 7 to 1. The break-even point is 4 to 1, because a quarter of 4 is 1, and we're aiming for at least 5 to 1 to keep some margin. So we don't lose the motor; we just plan the motor commands around a weaker one. These are estimates, and we'll measure the real numbers on a thrust test once the parts arrive.

Hand over: "Next, Jetty Jithendra will show where our code fits in the flight software."

Speaker 3 · Jetty Jithendra

Slides 5 · Architecture (flight stack) · about 1:00

Slide 5 – Architecture (3:00–4:00)

This is our architecture. A drone's flight software works as a chain. The waypoint mission says where to go next. The position controller decides which way to tilt. The attitude and rate controllers work out how much roll, pitch and yaw is needed. Then control allocation, also called the mixer, turns that into a speed for each of the four motors. We're building on PX4, an open-source autopilot, so we don't rebuild stable flight from scratch. The block we change is control allocation, shown in orange. On top of that we add three things: a fault interface that receives "motor k is at 50%", a fault-aware allocator that knows the new limit, and mission logic that decides what to do after the fault.

Hand over: "Next, Vineel Sai Reddy will explain how we reallocate the motor work."

Speaker 4 · Vineel Sai Reddy

Slides 6–7 · Control reallocation and fault interface · about 1:55

Slide 6 – Control reallocation (4:00–5:10)

This is the core of our solution. The mixer solves this equation: the controller asks for a total lift T and three turning moments, and the mixer finds four motor thrusts, f1 to f4, that produce them. Normally each motor can go from zero to its maximum. After the fault, motor k only reaches about 25% of its maximum thrust. PX4's default mixer doesn't know that. It keeps asking motor k for thrust it can't give, the motor saturates, and the drone tilts. We fix this in three steps. First, we give the allocator motor k's new limit so it plans around it. Second, we set priorities: lift, roll and pitch come first, and if something has to give, it's yaw, so the heading may drift slowly. Third, after the fault we lower the speed, climb-rate and tilt limits, so the controller rarely asks for more than the weak motor can give.

Slide 7 – Fault interface (5:10–5:55)

Next, how the fault reaches our controller. On demo day, the evaluator caps one motor at 50% RPM and tells our system which motor it is. We'll accept that either as a MAVLink command from the ground station or as an RC switch. Inside PX4 it becomes an internal fault message, and both the allocator and the mission logic switch to fault mode. While testing, we inject the fault ourselves: one switch on our remote caps the motor and sends the notification at the same time. Throttle isn't the same as RPM, so we calibrate the cap on the bench using the RPM readings from our ESCs. And everything runs in the PX4 simulator first.

Hand over: "Next, Hemanth will explain what the drone does after the fault, and our deliverables."

Speaker 5 · Hemanth

Slides 8–9 · After the fault, and deliverables · about 1:55

Slide 8 – After the fault (5:55–6:40)

After the fault, the drone decides on its own what to do. First, it receives the fault. Second, it holds position for a few seconds to settle. Third, it checks its health: is the tilt error small, is it holding position, and is there enough battery? If all three are yes, it flies the remaining waypoints at a lower speed. If any one is no, it flies back and lands. The brief accepts either ending; finishing the mission is what we're aiming for.

Slide 9 – Deliverables (6:40–7:50)

These are the nine deliverables we're committing to. D1 to D8 follow the project brief one to one, and the right column says how we'll prove each one. D1 is the drone itself, sized for the fault at a thrust-to-weight of at least 5 to 1. D2 and D3 are normal flight: a stable, tuned controller and autonomous waypoint navigation. D4 to D6 are the fault handling. That's the notification interface, then the fault-aware allocation, which we'll also compare against the default mixer in simulation, and then the controller reconfiguration after the fault. D7 is the pair of demo flights, one healthy and one with the fault. D8 is the decision after the fault: finish the mission or land safely. D9 is the flight logs, the data and the technical report.

Hand over: "Next, Shaik Babu will cover our scope and the airframe components."

Speaker 6 · Shaik Babu

Slides 10–11 · Scope, plus airframe and propulsion parts · about 1:10

Slide 10 – Scope (7:50–8:20)

To be clear about scope: these are stretch goals we'll try if time allows, but we're not committing to them. That includes finishing the full mission in every trial, a closed-loop RPM cap, and comparing the default mixer with ours in real flight, which we only commit to in simulation. Out of scope are fault detection, which the brief excludes, and other kinds of fault, like a fully stopped motor.

Slide 11 – Components: airframe and propulsion (8:20–9:00)

Now the components. For the airframe and propulsion, every choice follows the thrust-margin rule. A 5-inch carbon frame, because it's light, survives crashes, and spare arms are cheap. Four 2306 motors at about 2450 KV on a 4-cell battery, which gets us to about 7 to 1. A 4-in-1 ESC with RPM telemetry, so we can measure and log the real motor speed. Plenty of 5-inch props, because they break. And three batteries, so we get several test flights per session.

Hand over: "Next, Prasad will cover the control and safety components."

Speaker 7 · Prasad

Slides 12–13 · Control and safety parts, and closing · about 1:00

Slide 12 – Components: control and safety (9:00–9:40)

For control and safety: an H7 flight controller running PX4, because we need to change its allocation code and use its flight logs. GPS with a compass if the demo is outdoors, or an optical-flow sensor if it's indoors. A telemetry link to send the fault notification and watch the flight live. An RC transmitter with a kill switch, which the safety rules require, plus a second switch to turn the test fault on and off. And a tether rig with spare parts for the first tests. The total comes to roughly 45 to 55 thousand rupees.

Slide 13 – Closing (9:40–10:00)

Before we finalize our deliverables, we'd like to confirm four things: how the fault will be applied and signalled on demo day, whether the demo is indoors or outdoors, whether we can build on PX4, and whether the lab provides parts or a budget. Thank you. We're happy to take questions.

If they ask…

Short answers to the questions we're most likely to get. Anyone can answer; the person who presented that slide should go first.

Why PX4?

It's open source, it already does waypoint missions and detailed logging, and its control allocation is a separate module we can modify. That module is exactly what our project is about.

Isn't hover fine anyway at 7:1?

Yes, in calm hover the weak motor has enough. The trouble comes in climbs, turns, braking and gusts, when the controller suddenly wants more from one motor. The default mixer doesn't know the limit, so it saturates and the drone tilts. Our allocator plans around it, and we'll show the difference in simulation logs.

How do you know it's really 50% RPM?

Throttle and RPM aren't the same, so we measure the real RPM from ESC telemetry and log it in every test.

What happens to yaw?

We lose some yaw authority. We keep the drone level and on position first, and accept a slow heading drift.

Why not a hexacopter?

The brief asks for a quadrotor. A hexacopter has spare motors, which would make the fault much easier and miss the point.

Why not a bigger 450 mm frame?

To reach 5:1 on a frame that size you need much bigger motors and batteries, which costs a lot more. A 5-inch frame gets there cheaply and is easy to fix after a crash.

Why 4S and not 6S?

4S parts and chargers are cheaper and easier to find, and 4S already gives us enough thrust.

Why not a cheaper Betaflight board?

Betaflight can't fly autonomous waypoint missions and isn't built to be modified this way.

What are the health-check thresholds?

We'll set them from simulation and tethered tests, roughly a few degrees of tilt error and under half a metre of position error, and report the values we pick.

What if it can't hold position after the fault?

The health check fails and the drone returns and lands, which still meets the brief.

Tips

- Practise once with a timer. If the run takes 10:30, you'll probably finish around 10 minutes on the day, because most people speak faster when nervous.
- If you're running long, shorten slides 3 and 10 to one sentence each.
- Say numbers slowly and clearly, especially 25%, 7 to 1, and 5 to 1. They're the core of our argument.
- Deliverables can't change after today, so stick to the wording on slide 9 and don't promise extra results in Q&A.